

T430 → k3s cluster onboarding

joining an old ThinkPad as CPU agent node x2

A phased checklist to prep a scavenged **ThinkPad T430** (Ubuntu 24.04, WiFi, 100 GB disk, no GPU) and enroll it in the home k3s cluster as a **CPU agent** named x2. The catch: this drive is a **clone of a former GPU box** — it boots up thinking it's a3 and still carries a k3s install, a datastore, and NVIDIA/CUDA bloat. Two special cautions fall out of that, and they gate everything else. Follow the phases in order; the first two exist purely to keep this laptop from picking a fight with the real cluster.

Hardware ThinkPad T430 · Ivy Bridge · no GPU

OS Ubuntu 24.04 · WiFi (internal Intel)

Disk ~100 GB (cloned from a GPU box)

Role k3s agent · label box=x2

x2

TARGET NAME

x-series = CPU ThinkPads;
x1 already taken

4.33

RESERVED IP

192.168.4.33 · eero
reservation "t430"

8

PHASES

Router → purge → rename
→ prep → join → verify

agent

NOT A SERVER

No control plane, no etcd,
no Longhorn

Read these two before you touch the network

⚠ It thinks it's a3 — because it's a clone of a GPU box

The hostname is a3 because the drive is almost certainly a **clone of a GPU box (likely the old a3)**. It may still carry a full k3s install **plus the cluster's datastore plus a3's identity/certs**. If that k3s ever autostarts on the LAN it can fight the **real a3** (192.168.5.173) — duplicate control plane, split-brain datastore, corrupted cluster state. **Purging the old k3s is step 1**, and renaming the host is step 2. Do them before the laptop has any reason to reach the cluster.

Unlearn this: "add it as a masternode for HA"

Join as an **agent**, not a server. A second control plane means migrating a3 from **SQLite → embedded etcd (HA)**, and **etcd hates WiFi latency and wants fast disks** — an old laptop on WiFi is the *worst possible* etcd member (it would degrade the whole API, not add resilience). As an agent (like x1) it's ideal for light CPU workloads. HA is a real roadmap item, but it's better served by **wired desktops**, not a scavenged Ivy-Bridge ThinkPad.

The mental model: an agent is a kubelet + container runtime that phones home to the server's API at <https://192.168.5.173:6443> with a join token. It runs pods the scheduler hands it and reports status. It holds **no cluster state** — so if x2 dies, you lose only the pods pinned to it, nothing about the cluster itself. That's exactly why a flaky old laptop is a fine agent and a terrible server.

PHASE 0 Router — already done

The address is pinned so every later phase can address the node as 192.168.4.33.

- ☐ **DHCP reservation is set** in the eero: device nickname t430 → 192.168.4.33.
- ☐ The node will be named x2 in the cluster (set via `hostnamectl` in Phase 2).

Cross-subnet, and that's fine

The node sits on 192.168.4.x while the control plane is on 192.168.5.x — a different /24. This is fine: **spark already runs on 192.168.6.19 as a cluster member**, so the eero routes between these subnets. Just verify reachability before joining (Phase 5.5).

PHASE 1 ⚠️ Purge the cloned k3s CRITICAL

Before it touches the cluster. This tears out the inherited k3s install, datastore, and kubelet state so the clone can never present itself as a3.

```
# is there an inherited k3s install / datastore?
ls /usr/local/bin/k3s-uninstall.sh /usr/local/bin/k3s-agent-uninstall.sh 2>/dev/null
systemctl status k3s k3s-agent --no-pager 2>/dev/null | head

# run whichever uninstaller exists (server or agent), ignore if absent
sudo /usr/local/bin/k3s-uninstall.sh 2>/dev/null || true
sudo /usr/local/bin/k3s-agent-uninstall.sh 2>/dev/null || true

# wipe any residual config, datastore, and kubelet state
sudo rm -rf /etc/rancher /var/lib/rancher /var/lib/kubelet
```

Confirm it's gone: `which k3s || echo "clean"`

PHASE 2 Rename the host — kills the a3 clash

```
sudo hostnamectl set-hostname x2
sudo sed -i 's/\b(a3|old-hostname)\b/x2/g' /etc/hosts
```

Re-login or reboot so the prompt shows x2. k3s names the node after the hostname — it **must be x2 before joining**, or you get a second "a3" in `kubectl get nodes`.

Why Phases 1 + 2 come before anything network-facing

These two are the entire reason this isn't a normal onboarding. An inherited k3s **server** autostarting as "a3" on the same LAN is the one action that can damage the *live* cluster — a fresh agent join can't. Neuter the clone (purge) and strip its stolen identity (rename) first; only then is it safe to let the box talk to 192.168.5.173.

Reclaim the disk — purge NVIDIA/CUDA

There's no GPU in a T430, so every NVIDIA/CUDA package and every model cache the clone dragged along is dead weight on a 100 GB disk. Rip it out and reclaim the space.

```
# see what NVIDIA/CUDA cruft is installed, and which apt sources feed it
dpkg -l | grep -iE 'nvidia|cuda|cudnn' | awk '{print $2}'
ls /etc/apt/sources.list.d/

# removing these .list sources is what STOPS the recurring heavy downloads
sudo rm -f /etc/apt/sources.list.d/*cuda*.list /etc/apt/sources.list.d/*nvidia*.list
sudo apt-get update

# purge the driver + CUDA + cuDNN + container toolkit
sudo apt-get purge -y '^nvidia-.*' '^libnvidia-.*' '^cuda.*' '^libcudnn.*' 'nvidia-container*'
sudo apt-get autoremove --purge -y && sudo apt-get clean

# drop CUDA install trees + orphaned nvidia dirs
sudo rm -rf /usr/local/cuda* /opt/nvidia

# find the next big offenders, then trim the systemd journal
sudo du -x -h --max-depth=2 / 2>/dev/null | sort -rh | head -25
sudo journalctl --vacuum-size=200M
```

Check free space after with `df -h /`.

The apt .list source is the real fix

Removing the NVIDIA/CUDA apt .list source is what actually **stops the recurring heavy downloads** — purge the packages without it and apt upgrade will happily re-pull gigabytes of driver you'll never use.

Likely leftover cache/model dirs worth reclaiming

PATH	WHAT IT IS
~/.cache/huggingface	Downloaded model weights — often the biggest single win
/models · /opt	Ad-hoc model + tooling drops from the GPU box
/root/.cache	Root-owned caches (pip, hf) the clone accumulated
/var/lib/docker	Old container images/layers if Docker was used on the GPU box

PHASE 4

Node-prep scripts — the red-tape eliminators

Run these in order from the laptop repo (~/.git/home-cluster). Each one erases a class of friction that has bitten a node before. Order matters: sudoers first makes the rest unattended.

```
# from ~/.git/home-cluster on the laptop
for s in node-sudoers node-powersave node-sysctl node-hosts; do
  ssh brian@192.168.4.33 "sudo bash -s" < scripts/$s.sh
done
scp "$(mkcert -CAROOT)/rootCA.pem" brian@192.168.4.33:/tmp/lan-rootCA.pem
ssh brian@192.168.4.33 "sudo bash -s" < scripts/trust-land-ca.sh
```

SCRIPT	WHAT IT FIXES
node-sudoers.sh	Passwordless sudo for brian — do first; makes the rest unattended.
node-powersave.sh ★ laptop-critical	Masks sleep/suspend, sets lid-close = ignore, turns WiFi powersave off. Without it, closing the lid drops the node (this bit x1).
node-sysctl.sh	Raises inotify limits — the Jellyfin crashloop fix.
node-hosts.sh	Static harbor.lan → 192.168.5.173 — breaks the Pi-hole-outage image-pull deadlock.
trust-land-ca.sh	mkcert CA into the OS trust store + routes .lan DNS to Pi-hole; self-verifies against home.lan.

PHASE 5

SSH — bidirectional, passwordless

```
# laptop -> x2
ssh-copy-id brian@192.168.4.33

# make x2's own key and print its pubkey (for a1/a2/a3/x1)
ssh brian@192.168.4.33 'ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519 <<<y; cat ~/.ssh/id_ed25519.pub'
```

Paste that pubkey into ~/.ssh/authorized_keys on **a1/a2/a3/x1**. Verify with `ssh brian@192.168.4.33 sudo -n true && echo "keyed + nopasswd OK"`. Optional hardening once keys work: set PasswordAuthentication no.

PHASE 5.5**Reachability sanity check****run on the T430**

Because 192.168.4.33 is on a different /24 than the server, confirm the box can actually reach the control plane **before** joining. Run these on the T430:

```
ping -c2 192.168.5.173      # control plane reachable?
nc -vz 192.168.5.173 6443   # k3s API server
nc -vz 192.168.5.173 443    # harbor.lan / Traefik ingress
```

If any of these fail, sort the 192.168.4 ↔ 192.168.5 routing on the eero before continuing — the join can't succeed until the API server is reachable.

PHASE 6**Join k3s — as an agent**

Grab the join token from the real server, then install the agent pointed at 192.168.5.173:6443 with the **version pinned to match the server exactly** and the box label set at join time.

```
# read the current node-token off the server (a3)
ssh brian@192.168.5.173 sudo cat /var/lib/rancher/k3s/server/node-token

curl -sL https://get.k3s.io | INSTALL_K3S_VERSION=v1.35.5+k3s1 \
  K3S_URL=https://192.168.5.173:6443 K3S_TOKEN='<paste-token>' \
  sh -s - agent --node-label inference-club.com/box=x2
```

This is scripts/install-k3s-agent.sh with the version pin added so the agent matches the server exactly. **Do not record the token in this document** or anywhere in the repo.

PHASE 7**Harbor pull trust — after the agent is running**

```
scp "$(mkcert -CAROOT)/rootCA.pem" brian@192.168.4.33:/tmp/lan-rootCA.pem
ssh brian@192.168.4.33 "sudo bash -s" < scripts/k3s-registries.sh
```

Points containerd at Harbor's proxy cache + pins its CA, then restarts k3s-agent. Now x2 pulls docker.io/ghcr.io through Harbor like every other node.

PHASE 8**Verify — from the laptop**

- ☐ `kubectl get nodes -o wide` → **x2 Ready**, correct IP, v1.35.5+k3s1.
- ☐ `kubectl get node x2 -o jsonpath='{.metadata.labels}' | tr , '\n' | grep box` → inference-club.com/box=x2.
- ☐ Smoke-test a pull pinned to x2 (below).

```
kubectl run x2-test --image=harbor.lan/dockerhub/library/busybox \
  --overrides='{ "spec": { "nodeSelector": { "inference-club.com/box": "x2" } } }' \
  --restart=Never -it --rm -- echo ok
```

Foreseen red tape & gotchas

GOTCHA	HANDLING
Duplicate a3	Phases 1 + 2 handle it. If you ever see two a3's in <code>kubectl get nodes</code> , stop and delete the rogue one.
Lid-close = node death	<code>node-powersave.sh</code> is the fix; verify with the lid actually shut .
Weak, WiFi-only box	T430 WiFi is internal Intel (not a1's flaky USB dongle) so it should be reliable — but it's still WiFi + a weak Ivy-Bridge box with a 100 GB disk. Keep heavy/network-bound pods pinned elsewhere, don't enroll it in Longhorn , and use <code>nodeSelector</code> to steer big pods off it.
ufw	If enabled it blocks flannel VXLAN (8472/udp) + kubelet (10250). Check <code>sudo ufw status</code> ; disable on-LAN or open those ports.
inference.club registration	Not required for k3s. If wanted, replicate what x1's <code>inference-club-agent</code> did — a separate follow-up.
Clock skew	Verify <code>timedatectl</code> shows NTP synced; skew breaks TLS to <code>harbor.lan / .lan</code> .

Bottom line

Two facts about this drive dictate the whole procedure: it's a **clone that believes it's a3**, and it's a **weak WiFi laptop**. So the first job is defensive — purge the inherited k3s and rename the host *before* it can split-brain the live cluster — and the strategic call is to join it as a plain **agent**, never a control-plane member. Do that, run the five node-prep scripts (`powersave` is the one that keeps a laptop alive), pin the k3s version to the server, and x2 lands as a tidy CPU workhorse alongside x1 — extra scheduling headroom for light pods, zero new risk to the cluster's state.

Quick reference

THING	VALUE
Node name / role	x2 · k3s agent (CPU, no GPU)
IP (DHCP reservation)	192.168.4.33
Node label	<code>inference-club.com/box=x2</code>
Server / API	<code>https://192.168.5.173:6443 (a3)</code>
k3s version (pin to server)	<code>v1.35.5+k3s1</code>
Order of operations	router → purge k3s → rename → disk cleanup → node-prep → ssh → join → harbor → verify
Do NOT	join as server · enroll in Longhorn · pin heavy pods · commit the join token